University of Pisa

Ph.D. in Computer Science

Towards a green runtime support for parallel applications

Ph.D. Student

Daniele De Sensi

Supervisor

Prof. Marco Danelutto

Academic Year 2014/2015

Contents

1 Introduction **1**

2 State of the art **3**
2.1 Power and energy consumption 4
2.2 Available control knobs 6
2.2.1 Reducing frequency and voltage (DVFS) 6
2.2.2 Idle states 7
2.2.3 Concurrency throttling 9
2.3 Power and performance prediction 10

3 Proposal **11**
3.1 Structured parallel programming 11
3.2 Possible scenarios 13
3.3 Power aware runtime system 15

4 Work done in the first year **19**
4.1 API for mechanisms interaction 19
4.2 Runtime support prototype 21

5 Work plan **24**

6 Conclusions **26**

1 Introduction

In the latest years, increasing attention has been paid on the creation of power efficient computing system. This interest is motivated both by environmental and economical reasons. In fact has been estimated that, during 2010, data centers in the US consumed more CO_2 than that consumed by an entire country like Argentina or Netherlands [30] (and close to the CO_2 consumed by the worldwide airline industry). Furthermore, power consumption has also a consistent economical impact. In 2010, for example, the energy consumed in US by data centers reached the 3% of the overall energy production, and this power demand is estimated rapidly growing $10\% - 12\%$ a year [30, 26]. Consequently, energy cost in the near future could easily overtake the cost of the physical system itself. In addition to this, high power consumption causes high temperature in the system, increasing the probability of failure of the hardware components and requiring more power for advanced heat management and cooling systems. This often means that for every dollar spent on electricity, an additional dollar is required for cooling [38].

Moreover power efficiency is also needed on smaller scale, for example by mobile systems users, where power consumption is one of the main concerns. Finding efficient power management methods could lead to longer battery life and better user experience.

Albeit this problem can be addressed at various levels, we will focus on solutions involving individual applications. Hardware manufacturers provide different control knobs to cut down the power consumption. For example, the application programmer may decide to scale down the frequency of the CPUs or to turn off unused cores, caches or RAM modules, allowing thus to use just the resources really needed by the application [1]. In general, we have a certain number of n available control knobs $< k_1, k_2, \ldots, k_n >$. We call a specific combination $< v_1, v_2, \ldots, v_n >$ of the values of these knobs a *configuration*.

For example, we may have $n = 2$, k_1 equal to the number of cores used by the application and k_2 equal to the frequency of the cores. Consequently, the configuration $< 3, 1.4GHz >$ represents a situation when the application uses 3 cores at a frequency of $1.4GHz$.

However, it would be too cumbersome for the application programmer to manage these knobs since he should decide how many cores to use, how to

[1] Since a wrong utilisation of these mechanisms can have undesirable effects, their use is usually limited to users with advanced privileges.

allocate threads on the physical cores, at which frequency these cores must be run and so on. This is a complex task indeed, and wrong decisions may lead to a decrease in the performance and an increase in power consumption. Things get more complex if we consider the possibility to use such mechanisms inside parallel applications. Furthermore, the optimal configuration can be different for different inputs and, even specifying the input, it could change through time, both for external interferences but also for different characteristics between different phases of the application.

For these reasons, an increasing amount of literature is recently investigating the possibility for the creation of runtime supports that dynamically adapt themselves to satisfy some user requirements in terms of performances and power consumption, relieving the user of deciding those parameters or to manually change them during the application lifetime [40, 42, 28, 41].

These systems can cut power consumption in two ways:

- By shutting down components where they are not used. According to [38, 7] the average utilisation of many systems is usually around $10\% - 50\%$. This opens many possibilities for power saving by shutting down unused portions of the machine and increasing the average utilisation of these systems [18, 24, 40, 41]

- By slowing down the application. Indeed, by reducing its performances, a reduction in its power consumption may be achieved too. This set of solutions try to achieve a tradeoff between performance loss and power consumption reduction [15, 20, 29].

However, existing solutions have some limitations:

1. Many proposed approaches apply machine learning algorithms to predict the performance and power consumption of the application under different parameters. However, such approaches usually require long and complex training phases [37, 14, 15], which must be repeated from scratch if the application is executed on a different architecture. Moreover the goodness of these solutions often depends on the chosen training set and, when executed on an application too different from those used for training, the accuracy of the predictions may worsen and the optimal configuration could not be found.

2. Most existing works assume that once the optimal configuration is found, it will not change during the application execution [39, 37, 14]. However, in real world scenarios this is a restrictive hypothesis since

the conditions of the application could change, both due to perturbation in the underlying architecture but also due to phase changes in the application.

3. The results that are shown, are often obtained through simulations only [29, 39]. For this reason, is not always clear how those systems would react on a real environment.

Consequently, we would like to propose a general methodology for dynamic and power aware runtime adaptation of applications. This runtime system should solve the following problems:

1. It should avoid long training phases, by possibly embedding small and fast training phases directly during the application execution.

2. Be able to perform fast changes in the parameter of the application according to the monitored behaviour, while not significantly impacting on the performances.

3. It should be implemented and test on a real environment, in order to assess the validity of the approach on real world scenarios.

To achieve this goal, many theoretical problems must be faced, and we will describe them in detail, together with our thesis proposal, in Section 5.

The rest of this proposal is structured as follows. In Section 2 we will describe the state of the art of the research in this field. We will then present in Section 3 our thesis proposal, describing some of the algorithmic challenges we are going to face. In Section 4 we will briefly describe the work done up to now, showing a prototype we realised to assess the validity of our approach. Section 5 contains the workplan proposal for the next year. Eventually, in Section 6 we will make some considerations about this thesis proposal.

2 State of the art

In this Section, we present the state of the art in this research field. We first make a brief introduction on the concepts of power and energy consumption, showing the most important factors that affect them and how the available hardware control knobs can be used to modify these factors. We will then show the current advances by analysing some of the techniques currently used.

2.1 Power and energy consumption

First of all, it's important to point out that the focus of this thesis is on the power consumption of the CPU, since it is in charge of the greatest percentage of power consumption when the system is under load [22]. There are many existing works that analyse the power consumption of other components like disks, fans and network adapters. However, analysing these components is outside the scope of this thesis.

The power consumption is expressed in *Watts* (W) and, for the CPU, it can be defined as [11, 5, 25][2]:

$$P(\omega, \pi, \gamma) = P_{static}(\gamma) + P_{dynamic}(\omega, \pi, \gamma) = \gamma I_{leak} + \alpha c \gamma^2 \pi \omega \qquad (1)$$

where P_{static} is the power consumed by the CPU even when it is idle and not executing instructions and $P_{dynamic}$ is the power consumed then it is active. ω is the number of cores used by the application, π is the frequency of the cores on the CPU, γ is the voltage and c is the capacitance. I_{leak} is the current that leaks through transistors even when they are turned off (*leakage current*) and, for our purposes, we can consider it as a constant. α is the activity factor, i.e. the fraction of the circuit that is switching per unit of time and can be reduced by turning off the clock of unused circuits. Accordingly, we can reduce power consumption by decreasing the number of cores used by the application (ω), the frequency of the cores (π), the voltage of the CPU (γ) or by turning off the clock for unused components. However, all these techniques require a reduction in the amount of resources used by application thus causing a reduction in the performances. For this reason, we can divide power saving solutions between solutions which try to find a trade-off between power consumption and performances and solutions which exploit low utilisation of components to reduce the power consumption without harming the performances. Another way to reduce power consumption, is by executing the application on a different type of execution unit.

On the other hand, energy is defined as the power consumed on a time interval:

$$E(\omega, \pi, \gamma) = P(\omega, \pi, \gamma) \cdot T \qquad (2)$$

where T is the execution time of the application. Since cutting down the power consumption may increase the execution time, the best solution from

[2] If we consider a system with multiple CPUs, this formula applies separately for each of them.

Configuration	Power consumption	Execution time	Energy consumption
A	4W	4 sec	**16J**
B	**2W**	9 sec	18J

Table 1: Example of a situation where the best configuration in terms of power consumption does not coincide with the best configuration in terms of energy consumption. Albeit this is a synthetic example, this situation often occurs in many real world situations.

the point of view of the power consumption in many cases may not coincide with the best solution in terms of energy consumption. For example, by looking at the example shown in Table 2.1, we can see that Configuration A is the best solution in terms of consumed energy while Configuration B is the best in terms of power consumption.

At a first glance, it might seem that we should always consider energy consumption, since Configuration A is the one which delivers shorter execution time as well as a lower overall consumption. However, this came at the cost of an higher power consumption and in many cases this could lead to undesirable effects. For example, elevated power consumption raises the temperature of the system, increasing the probability of failure of the hardware components. Accordingly, advanced cooling equipment is required, which in turn consumes power and represent an additional economical cost. Since the effectiveness of cooling systems is based upon peak power, by reducing it we can reduce both the cost and the impact of the cooling equipment. Such kind of techniques are commonly known as *power capping* techniques [8, 15, 29, 20]. Moreover, many power suppliers provide price incentives for use of electricity during low demand or off-peak periods. In this case, *power capping* techniques could follow this trend in the prices by adapting the maximum allowed power consumption during the day to minimise the economic cost of electricity. Lastly, in some cases the application execution is driven by data availability. Consider for example an application R that receives data from another application S. In this case, even if R runs at the highest performance level, its execution time is determined by the speed at which S send the data. Accordingly, since R can't reduce its execution time by means of local decisions, the only possibility we have is to cut down its power consumption.

On the other hand, there are cases in which both power and energy consumption are considered, for example by finding the most energy efficient

solution under a maximum power cap [14]. However, since energy consumption can be derived from the power, for the rest of the proposal we will focus on power consumption.

2.2 Available control knobs

In this section, we present some commonly available mechanisms used to reduce power consumption. As we shown earlier, we can reduce the power consumption of the CPU by reducing its frequency and voltage, by shutting down unused portions of the CPU or by reducing the number of cores used by the application.

The possible mechanisms to operate on these knobs are defined in the Advanced Configuration and Power Interface (ACPI) standard [3]. This standard specifies different possible *power states* for the CPU, distinguishing between states where the CPU is actively executing instructions (P-states) and states where the CPU is idle (C-states). We will see in the next sections how these power states work and how they can reduce power consumption.

2.2.1 Reducing frequency and voltage (DVFS)

Currently available CPUs have different frequency operating points, each of them corresponding to a specific *P-state*. These states are denoted with P0 to Pn, with P0 being the highest performance state, whereas Pn is the lowest frequency operating point. By lowering the operating frequency, the supply voltage will be lowered too [9], thus allowing a reduction of both components of Equation 1. Moreover, the second component of the Equation decreases more than linearly with respect to the frequency, since a relationship exist between frequency and voltage. This mechanism is commonly known as *Dynamic Voltage and Frequency Scaling* (DVFS).

In the most general case, group of cores will share the same voltage, thus the operating point can't be changed individually for each physical core. We will denote this situation by saying that these cores belong to the same *voltage domain*. If different cores on the same voltage domain require different P-states, the selected frequency will be the highest among all the requests [23].

However, since the execution time is inversely proportional to the frequency [33], by reducing the operating frequency we also proportionally increase the execution time. Accordingly, since this relationship exists, DVFS

[3] `http://www.acpi.info/`

could be used to get a tradeoff between execution time and power consumption.

Anyhow, DVFS could also be applied without harming the performances by exploiting low utilisation periods of the CPU. Currently, the decision of when and how to scale the frequency can be made by the operating system by means of *governors*, i.e. algorithms which dinamically change the frequencies according to the current utilisation of the cores. Alternatively, such kind of decisions can be delegated to the application programmer.

When this technique was introduced, it was used to exploit waiting times when a memory request was issued [13]. To be precise, after requesting data from the memory and while waiting for the results, the frequency was scaled down by the application to reduce its power consumption. However, the effectiveness of this technique on current architecture is reduced [27] due to improved memory performances and to the presence of smart operating systems governors that already perform such kind of operation.

Accordingly, as we will show later, DVFS is now mostly used in conjunction with other mechanisms [18, 29, 15].

2.2.2 Idle states

States that reduce power consumption of the idle system are also referred as *C-States*. They are defined from C0 to Cn, with higher index corresponding to increased power reduction. Going from C0 to Cn, different part of the CPU are turned off, allowing thus to save energy. In Table 2.2.2 we describe the physical cores C-states for an Intel Haswell architecture. These states can slightly change on different architectures, i.e. highest states may not be present or additional states may exist. However, states from C0 to C3 are defined by the ACPI standard and are present on almost all new generation architectures [12].

Resolution of C-states occur at the virtual core, physical core and CPU level, in a hierarchical way. For example, a physical core C-state is determined by the lowest numerical C-state of its virtual cores. If virtual core 0 requests C1 state while virtual core 1 requests C3 state, the physical core will stay in C1 state. Similarly, a CPU C-state is determined by the lowest numerical C-state among all its physical cores. Moreover, while physical cores C-states reduce power consumption of the elements belonging to the core, package C-states reduce the consumption of uncore components. For example, in package C6 state, L3 cache is flushed and its voltage reduced to 0V (*power gating*). However the highest will be the C-state, the highest will be the latency to enter and exit from this state.

State	Description
C0	Core executing code.
C1	Core is halted.
C1E	Core is halted and set to the lowest frequency.
C3	Core flushes L1 instruction cache, L1 data cache, and L2 cache to the L3 shared cache. Clocks are stopped (*clock gating*).
C6	Architectural state is saved to a dedicated SRAM. Voltage reduced to 0V (*power gating*).

Table 2: Intel Haswell cores C-States description.

Similar to P-states, C-states are managed by the operating system by means of governors, which decide when to enter in a deepest idle state. Usually, a core enters into an idle state when it predicts that the core will stay inactive for a sufficiently high amount of time. The longer is this amount of time, the deepest will be the idle state entered. Even in this case, the application programmer can decide to directly interact with them, for example by disabling some of these states.

C-states are mainly designed to operate without harming the application performance, by turning off unused portions of the CPU only when there is no computations to perform. However, in some cases [20] they are used to achieve trade-off between power consumption and performances. For example, let's assume that an application must not consume more than 170 Watts and that when the CPU is idle it consumes 120 Watts while when it operates at the highest frequency it consumes 220 Watts. Then, the application could alternate between idle and maximum frequency states, spending half of the time in each state. This will of course cause a performance degradation, since the application could always run at maximum frequency. However guarantees on maximum execution time can be provided.

Another interesting scenario is the one where idle states are used together with frequency scaling. Consider for example the case where the application must respect some time deadline. Then, we have to possible ways to execute the application. The first one is commonly known as *race-to-idle* and consists in running the application as its maximum performances to finish at soon as possible and then put the CPU into an idle state. The second solution, also known as *exploiting slack*, is to spread the execution such to finish exactly at the deadline time, by slowing down the application. These two possible situations are depicted in Figure 1.

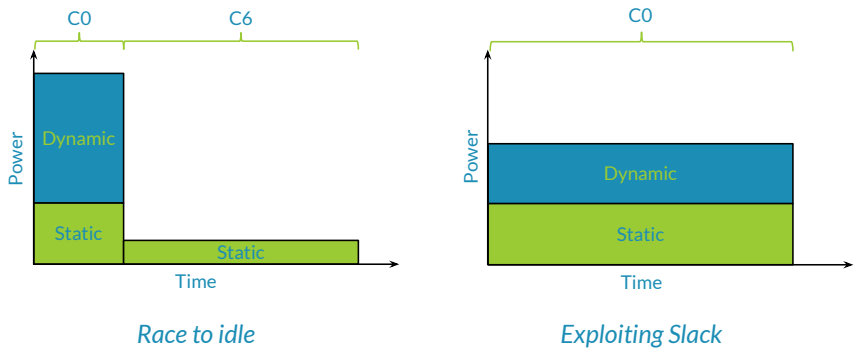

Figure 1: Comparison between *race-to-idle* and *exploiting slack*.

As we can see, in the first case we consume an higher amount of static and dynamic power while the application is running, but then for the rest of the execution we will only consume a low amount of static power, since when the CPU is idle the dynamic power is zero. On the other hand, in the second case we consume a lower amount of dynamic and static power but for longer time. The best solution in terms of average power consumption usually depends on the ratio between static and dynamic power. More importantly, the best solution can be different on different architectures [33].

2.2.3 Concurrency throttling

Another possibility we have to cut down dynamic power consumption is by using a lower number of cores. This technique is commonly known as *concurrency throttling*. As common in the parallel computing community, we assume a situation where we execute at most one thread for each physical core on the CPU. Accordingly, to reduce the number of used cores we will reduce the number of threads used by the application. Other existing solutions [15] work by moving more threads on few cores instead of reducing their amount.

Concurrency throttling has been extensively studied by the behavioural skeletons community [4] as a viable way to adapt the used resources to the real application needs. Recently, an increasing amount of literature investigates the relationship between concurrency throttling and power or energy consumption.

Some of these works [31, 36, 10] are only targeted towards energy consumption reduction. Moreover, since they assume that to a shorter exe-

cution time correspond a lower energy consumption, they do not explicitly estimate power consumption, thus not allowing trade-offs between power and performances.

On the other hand, other solutions [18, 29, 43, 14] try to estimate both power and performances, in order to be able to reduce power consumption. Often, such kind of solutions propose mixed approaches between concurrency throttling and other power reduction techniques.

However, the main problem of this technique is that it is often difficult to implement. For this reason, many existing approaches are only simulated [29, 10, 14, 39]. In other cases, due to limitations of existing runtime supports, the number of threads can only be changed between different parallel regions [31], thus limiting the effectiveness of the approach. As we will see later, one of the targets of this thesis is to overcome such limitations.

2.3 Power and performance prediction

To be able to decide the best configuration in terms of power and performances, existing solutions must be capable of predicting the behaviour of the application in all the possible configurations. For example, they must be able to predict how execution time and power consumption of the application are affected by a change in the number of threads or in the frequency of the cores.

First of all, it's worth noticing that a prediction method is necessary since a full exploration of all the possible configurations would be unfeasible. Indeed, their number is proportional to the product of the possible values for each control knob. For example, if we have 24 cores and 13 possible frequency levels, we have 312 possible configurations. Executing the application in all these configurations to find the best one would have a tremendous impact on the performances.

Accordingly, existing solutions usually explore only few of these configurations and then try to predict the behaviour of all the others.

Some works [29, 37] use heuristics to cut down the exploration space and to reduce the number of configurations to be explored. Other approaches [39, 18] apply analytical models to infer the optimal configuration.

More complex solutions [21, 17, 14] apply interpolation or machine learning solutions to train the algorithm on some explored configurations. After the training phase, they are able to predict the behaviour of the application in different configurations. However, as we will show soon, we would like to perform this exploration phase while the application is running. Accordingly, these techniques should be simple enough to do not significantly

reduce the performances of the application.

3 Proposal

In this Section, we will better outline our thesis proposal and the main goals we would like to achieve, showing some of the challenges that we are going to face and how they could be solved.

As we anticipated in Section 1, existing solutions have some limitations, that we would like to overcome by proposing a methodology for a unified runtime management of power related mechanisms. In the following Sections, we will define some guidelines to be followed to achieve this goal.

3.1 Structured parallel programming

Firstly, we should define which type of applications can benefit of such approach.

Any parallel application can be expressed as a graph of concurrent activities cooperating to reach a common objective. However, among all the possible graphs that can be used to represent an application many real applications can be described using some *structured* and recurrent schemes called *skeletons* [16, 2, 6]. Skeletons are programming patterns which allow to model typical forms of parallelism exploitation, expressed in parametric form. They have a precise semantic and are characterised by a specific cost model, which can be used to evaluate their performance metrics. Moreover, they can be composed and nested together to represent more complicated parallelism forms. Since developing an efficient parallel application is often a difficult task, when structured parallel programming methodology is used, it's possible to hide a part of this complexity and to let it be managed by the library or the programming language used to describe the application [3, 6].

Our work will be targeted towards structured parallel applications for the following reasons:

- They are **parametric** with respect to the number of threads, thus by changing the parallelism degree we will not change the **semantics** of the application.
- Due to their well defined **cost model**, we are able to predict their behaviour in terms of performances and power consumption when the parameters of the application are changed.

- Since one of our target is to hide the complexity of the power management techniques to the application programmer, this perfectly fit with the founding principles of **structured** parallel programming.

To better understand the reasons of this choice we now describe one of the most known and used patterns in parallel application, the *farm*. Moreover, this is the pattern on which we would like to focus in the first part of this thesis.

A farm is a paradigm based on functional replication, and which can be represented by the graph depicted in figure 2. It is composed by: a function replicated into a set of modules $\{W_1, W_2, W_3 \ldots, W_N\}$ called *workers*, an *emitter* E which schedules the input data to the workers and a *collector* C which collects the produced results. Therefore, in general, the function will be applied at the same time by different workers over different input elements.

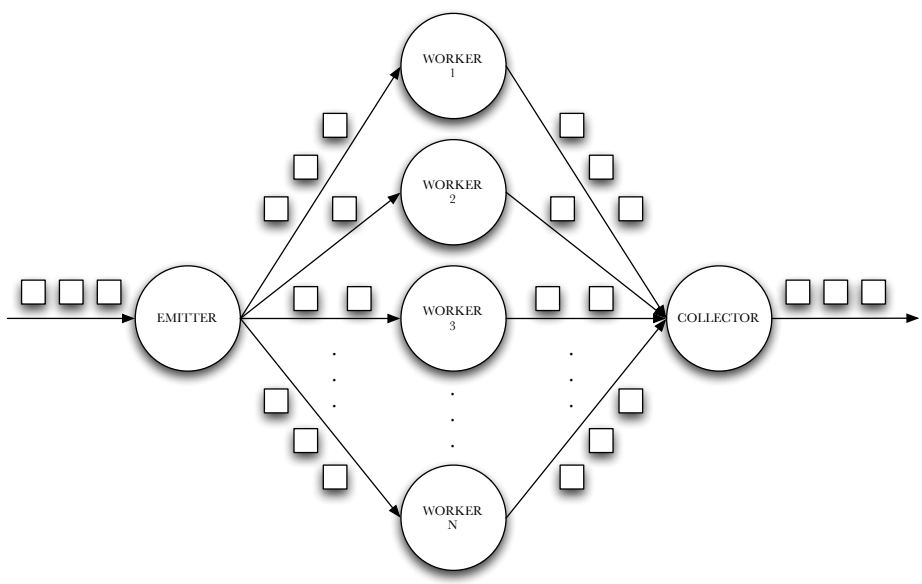

Figure 2: Farm skeleton

Different scheduling strategies may be used by the *emitter*. The most common are:

Round robin This is a simple strategy which sends the input data circu-

larly to the available workers. Since each data element may require a different amount of time to be processed, this strategy does not provide any guarantee on the balancing of the workload between the workers.

On-demand Using this strategy is possible to maintain a good balancing among the different workers. It could be implemented, for example, by allowing each worker to communicate to the emitter when it finished to process the data element and is thus ready to accept a new one.

Although the *on-demand* strategy may guarantee some kind of load balancing between the workers, it is more costly to be implemented with respect to *round robin*. Accordingly, when the computation is already balanced, it may be more convenient to use a *round robin* scheduling. However, since the balancing of the computation often depends on the input data, it may be unfeasible to statically decide a strategy which will be the best one in all the cases. For this reason, one of the problems our runtime support could try to solve is the dynamic choice of the best scheduling strategy according to the current balance of the computation.

One of the reasons why we focus on *structured* parallel applications is due to its parametric and semantically defined behaviour. Indeed, if we consider the farm, we can change its number of workers while the application is running without changing its semantics. This is always true if the function computed by the workers is a pure function. If the function has a state, it is still possible to dynamically change the number of workers but the state may be redistributed among the workers, as we shown in [18].

The other reason for choosing *structured* applications is their well defined cost model. As a first approximation, we can define the execution time $T(n)$ of a farm with n workers as $T(n) = \frac{T_{seq}}{n}$, where T_{seq} is the sequential time of the application. Similar models can be derived for the power consumption. By using such knowledge, we can correctly choose the best number of workers that satisfies some conditions in terms of execution time and/or consumed power.

3.2 Possible scenarios

To better understand the need and the potential of the technique we are going to propose, we would like first to introduce some possible usage scenarios for our runtime support. Each of these scenarios corresponds to a possible requirement that can be specified by the user of an application implemented by using our runtime support.

Maximisation of the system utilisation Many applications are characterised by high workload variations during its execution. Consider for example a network monitoring application. In this case, the workload of the application is mainly dependent on the bandwidth of the data travelling over the network. Such traffic is usually characterised by some peak periods and other low traffic periods. Since the monitoring application must be always able to manage the input traffic, it is usually dimensioned to manage the peak rate, wasting resources in the remaining periods [19]. To solve this problem, one commonly used approach is to dinamically reduce and increase the resources allocated to the application according to the real requirements. However, when multiple control knobs are available, multiple configurations may be able to manage the input bandwidth. However, among all these possible configuration, we want our runtime system to be able to choose the one with lowest power consumption.

Power capping As we described in Section 2, one common requirement is to limit the maximum power consumption of an application. However, due to the presence of multiple control knobs, we have multiple ways to cut down the power consumption. Since we would like to have a minimum impact on the application performances, our runtime system should be able to choose the most performing solution among those available.

Execution time capping In some situations, it may be useful to do not run an application at its full performances but still to have some guarantees on its maximum execution time. Consider for example a nightly backup system. In this case, we are only interested that the backup finishes before 8a.m. in the morning. Accordingly, our proposed approach could slow down the application in order to cut down power consumption, but still to be able to terminate before the deadline. Again, since there are different possible configurations that satisfy this condition, the runtime system will choose the one with lowest power consumption.

Derived metrics Other metrics can be derived from those already described. The most intuitive case is the one of the energy consumption. Indeed, since it is the product between execution time and power consumption, the previously proposed scenarios can be easily adapted to consider energy consumption instead of power. Other more complex metrics could be CO2 emissions or economical cost (both related to energy consumption).

Dynamic metrics Moreover, we could have scenarios where the metrics change over time. For example, let's consider an economical cost minimisation scenario. Since often the electricity cost changes during the day, we can imagine a situation where we have an higher power budget when the energy cost is lower and a lower power budget when the energy cost is higher.

Multiple metrics Up to now, we only considered at most two metrics at a time, like power minimisation under execution time capping or execution time minimisation under power capping. However, in many cases it could be useful to consider more metrics at a time. For example, we may want to minimise energy consumption within a certain power constraint and a maximum execution time.

In principle, our approach would be able to manage all of these scenarios, as we will show in Section 3.3.

3.3 Power aware runtime system

As we shown earlier, due to the increasing number and complexity of control knobs available for power consumption reduction, statically finding the best combination of these knobs is unfeasible. For this reason, in this Section we propose a runtime methodology to dynamically find and adapt the knobs values while the application is running. Our approach will be able to manage all the scenarios depicted in the previous Section by dividing the execution in two possible phases: *calibration* phases and *steady* phases.

Our runtime enters the calibration phase by starting the application in a given configuration. Then it proceeds in the following way:

1. The performances and power consumption of the application are monitored for a short period of time.

2. We refine our prediction algorithm by adding these new data. Then, we use it to predict performances and power consumption of the application in all the other possible configurations.

3. We choose a new configuration according to some strategy. The application is then moved to the chosen configuration.

4. The predicted values are compared with the actual ones. If the prediction error is lower than a specified threshold the calibration phases finishes. Otherwise, the process is iterated. This step is based on the

assumption that the error is uniformly distributed over the configurations. Accordingly, by looking at the prediction error on the current configuration we have an estimation of the error in all the possible configurations. As we will see in Section 4, this is a reasonable assumption, satisfied by some prediction techniques over many real applications.

Essentially, we are training a machine learning algorithm while the application is running. When a calibration phase finishes, we have an available model capable to predict performances and power consumption of the application in all the possible configuration with high accuracy. Accordingly, the runtime support predicts the best configuration according to the specified requirements and move the application in that configuration, thus entering in the steady phase.

While in steady phase, the runtime support will keep monitoring the application and, if needed, it changes the configuration of the application to react to workload intensity changes. In some cases, it may need to recalibrate the performance and power consumption models. For example, let's consider the case where the application finishes a CPU intensive phase and enters into an I/O intensive phase. In this case, the models computed for the CPU intensive phase will not be valid anymore and the runtime support will need to recompute them. Accordingly, it repeats the calibration phase to obtain the new models.

In our methodology, we can imagine the steps we described as a framework. The details of this framework can then be specified by means of *strategies*. A strategy is an algorithmic procedure that address a specific detail of our approach.

For example, it's important to notice that during the calibration phases the application is running and processing data elements and so when switching from calibration to steady phase we will not lose the results computed up to that moment. However during calibration phases is highly probable that the application violates the requirements and, for this reason, calibration phases should last as short as possible. This poses significant algorithmic challenges, each of them corresponding to a *strategy*. At the moment, we individuated the following:

Prediction strategy The first way we have to reduce the cost of the calibration phase, is by visiting the least possible amount of configurations. Since this amount is determined by how rapidly we can achieve an high prediction accuracy, this mainly depends from the prediction strategy used. However, it's worth noticing that we are training the algorithm on the same

application we are trying to predict. Accordingly, as we will show in Section 4, an high accuracy will rapidly be achieved.

Calibration strategy Another point we have to solve, is how to choose the configurations to visit in the calibration phase. We can imagine a n-dimension configuration a point on an n-dimensional space, where each dimension represent a control knob. An important observation is that if two points are close to each other, then we are collecting data about two configurations which are very similar, i.e. they have similar control knobs values. In this situation, most likely the second point will not add any significant information since they will most likely have similar performances and power consumption. Consequently, the more evenly distributed the points are, the more information they provide to the algorithm, thus allowing an higher accuracy. For this reason, we generate these points by using *low discrepancy* generators [35]. Such generators cover the domain more evenly with respect random or pseudo-random generators, as shown in Figure 3.

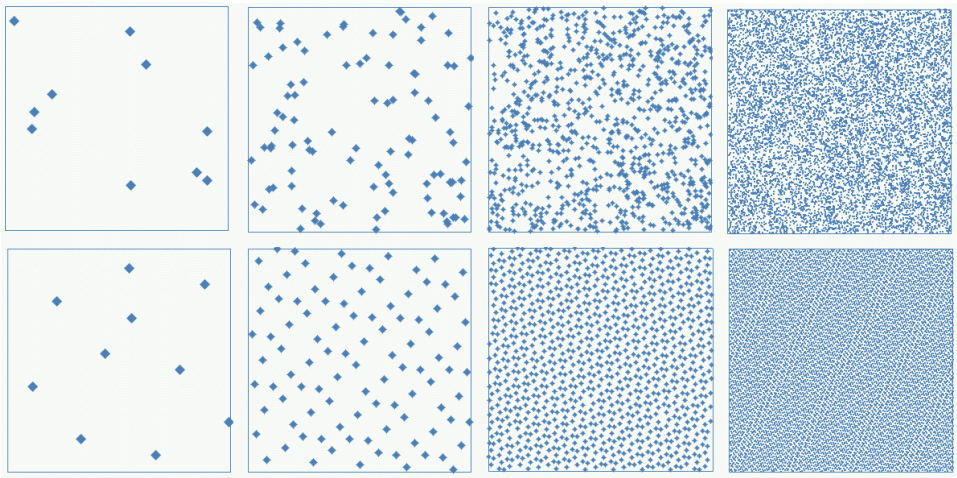

Figure 3: Comparison between the points generated by a pseudorandom generator (top) and those generated by a low discrepancy generator (down). From left to right, we have 10, 100, 1000 or 10000 generated points.

Moreover, they also have an advantage over deterministic methods since the latter give good equidistribution properties only when the number of points is known a priori, while in low discrepancy generators the distribution improves as more points are added. This is particularly suited to our approach since we visit different configurations until the accuracy is suffi-

ciently high, and thus we do not know a priori how many configurations we are going to generate.

Persistence strategy Another parameter that influences the cost of the calibration phase, is how long the runtime support stays in each visited configuration. The problem here is that we need to stay in a configuration for a period of time long enough to understand the behaviour of the application in that configuration but not so long to hurt the performances. One possible solution would be to consider the last S samples and to change configuration when the variation among the past samples is small. Although this may work for stable computation, we need to find better strategies for applications with high variability.

Recalibration strategy Since a calibration is done each time we detect a change in the intrinsic behaviour of the application, our method should be sensible enough to correctly adapt to changes. However it should not be too sensible in order to avoid temporary fluctuations, causing the system to be unstable. Up to now we assumed a *reactive* approach, i.e. we take the decisions after that changes occur. However, more advanced techniques based on *proactive* approaches could be used. This class of techniques try to predict the behaviour of the application in the future and, by using control theory techniques, allows the runtime system to avoid unnecessary recalibrations and to better react to workloads changes [32].

More strategies are related to other aspects of the approach. For example:

Reconfiguration strategy Performing a reconfiguration may be a costly operation, especially if the reconfiguration involves a change in the number of threads and a redistribution of the state of the application. Consequently, we would like to avoid unnecessary reconfigurations due to temporary fluctuations in the workload conditions. A first solution to this problem, could be the application of *smoothing* techniques. For example, instead of considering instantaneous values to detect changes, we could consider an average over the last S samples. In this way, we will avoid reconfigurations due to temporary spikes in the data. A more advanced solution could involve also in this case a *proactive* approach, similar to what we have seen for recalibration strategies.

Sampling strategy In order to detect changes, in the steady phase the runtime support will constantly monitor the behaviour of the application. Usually, this is done by sampling the performance and power consumption values at regular intervals. However, choosing the most suitable sampling interval duration may be a complex task. Indeed, with a low sampling interval the runtime support would always stay active and computing, thus increasing the overall power consumption and possibly also interfering with the other threads of the application. On the other hand, with an high sampling interval we increase the reaction delay of the support to changes in the workload. Ideally, we would like to have a low sampling interval for applications characterised by an high variance in the workload and an high sampling interval in all the other case. Consequently, we can imagine a situation where the sampling interval duration dinamically changes depending on the workload characteristics.

As we have seen, finding the best strategies may be a difficult problem. Moreover, we would like not to leave this choice to the application programmer, otherwise we would have moved the problem from choosing the knobs values to choosing the most appropriate strategies. Accordingly, we want to find strategies which are general enough to work in the majority of the cases without any need of intervention by the programmer. This is probably the most difficult target to achieve and we will most likely need to study and adapt approaches borrowed from other research field.

4 Work done in the first year

In this Section we will outline the first steps we did to reach our goal, by showing the prototype we realised and by analysing some of the results we obtained.

4.1 API for mechanisms interaction

Firstly, we needed some programming mechanisms to easily interact with machine related aspects and to easily modify them. However, we were not able to find a solution completely suited to our purposes. Accordingly we wrote Mammut[4], a C++ programming API that provides an object oriented abstraction of the underlying architecture and of its mechanisms. By using this library, we can easily analyse the topology of the underlying archi-

[4] `https://github.com/DanieleDeSensi/Mammut`

tecture, correlate the topology with voltage domains and power consumption information and manage the processes and threads running on the machine. More important, since many of these mechanisms can only be accessed by privileged user, we provided the possibility to run the library in a client-server fashion. In this way, we can run the server with privileges and let a set of non-privileged clients to interact with the server to access these mechanisms.

To better understand the expressiveness of this API, we provide the following code snippet:

```
1  Mammut m;
2  Topology* t = m.getInstanceTopology();
3  TasksManager* tm = m.getInstanceTask();
4  CpuFreq* f =  m.getInstanceCpuFreq();
5  Energy* e = m.getInstanceEnergy();
6  // Read topology information
7  vector<Cpu*> cpus = t->getCpus();
8  cout << "We have: " << cpus.size() << " CPUs" << endl;
9  // Map the application on a core
10 PhysicalCore* targetCore = cpus[0]->getPhysicalCore();
11 ProcessHandler* p = tm->getProcessHandler(getpid());
12 p->move(targetCore);
13 // Change the frequency of the core used by the application
14 Domain* frequencyDomain = f->getDomain(targetCore);
15 frequencyDomain->setFrequencyUserspace(1.8GHz);
16 // Run the application and get its energy consumption
17 CounterCpu c = e->getCountersCpu(targetCore);
18 JoulesCpu before = c->getJoules();
19 // Real application code
20 JoulesCpu after = c->getJoules();
21 cout << "Joules consumed: " << (after - before) << endl;
```

As we can see, with minimal effort, we are able to understand how many CPUs our machine has (lines 6-8) and to map our application on one of the physical cores on the first CPU (lines 9-12). Then, we can change the frequency of the core where the application is running (lines 13-15). Eventually, we let the application run and we measure its energy consumption (lines 16-21).

This allowed us to develop our prototype and to implement some of the strategies previously defined. To be used in a client-server way, we only need to change the first line, which would become:

```
Mammut m(CommunicatorTcp(address, port))
```

where *address* and *port* are the IP address and the TCP port used by the server.

4.2 Runtime support prototype

To assess the validity of of our proposal, we extended an existing parallel programming framework by adding our runtime techniques for power efficient reconfigurations. For our purposes, we chosen FastFlow, a C++ parallel programming framework providing the application programmer with customizable and efficient parallel patterns for shared memory multi-core platforms [3]. FastFlow provides different, fully customisable and composable patterns including a *farm* skeleton. The main reason why we chosen FastFlow is because it provides the possibility to "pause" the applications and dynamically add or remove threads. Accordingly, this will allow our runtime support to dinamically change the number of cores used by the application.

The first version of the support has been presented at PDP 2015 conference in Turku [18]. To assess the validity of our approach, we implemented the support on a Deep Packet Inspection application we realised in a previous work [19]. Using our runtime support, we were able to adapt the resources used by the application to the bandwidth of the data over the network, as shown in Figure 4, thus proportionally reducing its power consumption.

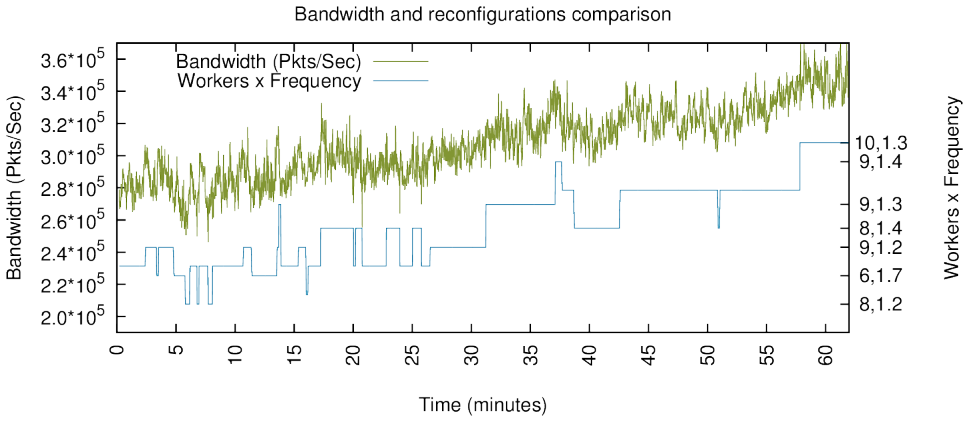

Figure 4: Comparison between input bandwidth to the application and allocated resources.

However, this version had some limitations. Firstly, we didn't have cal-

ibration phases and, to perform prediction, we used a simply analytical model. This model was not able to provide an exact estimation of the power consumption of the application. Nevertheless, it was able to provide estimations that respected the relative order between configurations. To be precise, we were able to determine that a configuration consumed less power than another one but we weren't able to determine the exact amount of power in the two cases. At that time this was sufficient for our purposes since it allowed us to manage all the input data with the lowest power consuming solution. Concerning the performance model, we were able to estimate the utilisation of the system with an error $< 4\%$. However, the model assumed that the application would always linearly scale with the amount of used resources, thus introducing an high error in all the other cases.

Currently we overcame all these limitations by introducing the calibration phase. To derive an accurate model for both performances and power consumption we used a multiple linear regression approach [34]. As we shown in Section 3.3, while the application is running we explore some possible configurations collecting data about performance and power consumption. We then interpolate these data in order to find analytical models. When the error between predicted and actual values is lower than a specified threshold the calibration finishes and the application enters in a steady phase. As we already discussed, we need to explore the minimum possible number of configurations. Accordingly, we need to verify if a multiple linear regression model is capable of achieving such result in this scenario. For this reason, we applied this model to the applications of Parsec, a well known benchmark containing applications from many different domains and with different characteristics in terms of parallelization model, working set size and data sharing and exchange between threads, thus allowing to test our approach over a wide range of real world scenarios. We tested this approach on a machine with 312 possible configurations (13 frequency levels times 24 physical cores), showing that in average we were able to achieve an accuracy of 96% for both performance and power consumption by only exploring ~ 4 configurations ($\sim 1\%$ of the total). However, as anticipated in Section 3.3, to achieve such target we carefully need to choose the configurations to explore. For this reason, we compared the accuracy of the method when the configurations are randomly chosen with the accuracy when low discrepancy generators are used. Our results shown that, in average, if we randomly generate the configurations we need 20 configurations to achieve a 96% accuracy, against the 4 configurations needed when they are generated with low discrepancy techniques. This means that, if we consider to stay in each configuration for a constant time, by randomly generating the

configurations to visit we would spend 5 times more time in the calibration phase with respect to the case in which we generate the configurations by using a low discrepancy generator. This clearly depicts the type of theoretical challenges we are going to face during this thesis. This study has been submitted to PDP 2016 conference and is currently under review [21].

At the moment, the control knobs we support are the number of cores used by the application and their frequency. Concerning the strategies, we implemented the multiple linear regression prediction strategy, the low discripancy strategy for the generation of configurations to be explored and two smoothing strategies (moving average and exponential smoothing) for the reconfigurations. All the others strategies still require some manual intervention from the programmer for fine tuning. The support is implemented by means of an external thread that we call *manager*, which monitors the application and reconfigures it when needed.

The current version of the support allows to express power efficient farm computations in the following way:

```
#define MAX_NUM_CORES 24
struct Emitter: ff_node{
    void* svc(void*){
        // Generate a data element t
        // (e.g. by reading a packet from the network)
        return t;
    }
};

struct Worker: ff_node{
    void* svc(void* t)
        // Compute something on t
        return t;
    }
};

int main (int argc, char* argv[]){
    vector<Worker*> w;
    for(uint i = 0; i < MAX_NUM_CORES; i++){
        w.push_back(new Worker);
    }
    ff_farm<> f(workers);
    f.add_emitter(new Emitter);
```

```
24
25      AdaptivityParameters ap("parameters.xml");
26      ManagerFarm amf(&f, ap);
27      amf.start();
28      amf.join();
29  }
```

In this example, we first define a thread that generates the data elements to be processed, for example by reading packets from the network (lines 2-8). Then, we specify the function to be executed by the workers of the farm (lines 10-15). The farm is created by specifying the maximum number of workers to be used (lines 18-22) and by adding the threads that generates the elements (line 23). Hence, we create the manager by specifying the application it must monitor and some optional user defined parameters (25-26). These parameters can be specified by using XML files to limit the knobs that the manager will use or to force the manager to use some specific strategies. Since they are optional, if not specified the manager will use all the available knobs and the best strategies for each case. Eventually, we start the manager and we wait for the application termination (lines 27-28).

5 Work plan

In this Section we will try to outline a long term work plan, specifying which points we would like to target during the next year. What we have shown in Section 4 will serve as starting point for the exploration of different techniques, which we depict in this Section. We would like to point out that some central issues of our approach are still to be analysed and solved and by proceeding in our study we could discover points that would radically change the situation.

However, the work can proceed simultaneously in many different directions. Here we propose some of them, from those with higher priority to those with lower priority.

1. Stability of the system. To achieve this goal, we need to find good algorithmic strategies for the calibration phase as well as for the reconfigurations in the steady phase.

2. New control knobs. For each possible knob we should proceed in the following way:

(a) Individuate a key parameter which we want to be automatically tuned by the runtime support (e.g. cache disabling, mapping of the threads on the physical cores).

(b) Understand how performance and power are influenced by this parameter over a wide set of significant applications (e.g. PARSEC benchmark).

(c) Try to find a model which include both the new knob and the old ones.

(d) Plug the new knob into the runtime system and assess it.

3. More programming models. So far we only discussed the case of farm based computations. However, it can be easily extended to data parallel applications (e.g. map-reduce models). With more effort and by introducing new techniques this approach could also be extended to other computational models like pipeline or task based models (e.g. Macro Data Flow [1]).

4. Consider latency. In some cases, the user may need to specify bounds on the maximum latency allowed for the processing of the single element. This is a common requirement, which would require to apply queueing theory techniques to be solved.

5. Distributed memory. Up to now we only considered shared memory systems. However, with appropriate improvements, this approach could be extended to distributed systems.

6. Multiprogrammed environment. Instead of considering only single applications in isolation, we could consider to apply this approach to many running applications at the same time. This poses significant challenges since the resources will then be shared and some sort of coordination will be required among the applications. This is particularly true on distributed and possibly heterogeneous systems, where the type and amount of resources differ and bigger algorithmic challenges will be faced.

7. Change the structure of the application. In some cases, it is possible to change the structure of the application by apply *rewriting rules*, i.e. by trasforming the parallel structure of the application into a semantically equivalent one but with different characteristics in terms of performances and power consumption [1].

In the next year, we would like to focus on the first two points. Concerning point 2, we already individuated some possible features, like:

- Mapping of the threads on cores. For example, we may want to map the threads on the CPUs such to have as few shared caches as possible, in order to reduce the number of memory accesses for each thread. This solution will try to spread the threads on the machines such to use as much caches as possible. An alternative solution, could be to use as less caches as possible and to disable the unused ones, in order to reduce the overall power consumption of the system.

- Use of hyperthreading. Not all the applications benefit from the use of hyperthreading. Therefore, in some cases we may want to use a number of threads not greater than the number of physical cores. This would have a benefit not only on the execution time but also on the energy consumption.

- Grain of the computation. In general, increasing the grain increases the bandwidth but also the latency. Accordingly, we may want to dinamically tune it in order to satisfy the requirements.

- Scheduling of the tasks to the workers. We could dynamically change the scheduling type according to the current unbalance of the application, in order to have an always balanced computation.

- Cache disabling. Since the power consumption of caches highly contributes to the overall consumption [44], we could disable the caches for computations with low data locality, which would almost not use the caches since the majority of access will be done in memory.

6 Conclusions

In this document, we proposed a general methodology to build highly dynamic runtime supports. By using this methodology, we should be able to adapt a parallel computation in order to respect some requirements needed by the user, both in terms of performances and power consumption.

We believe this is a challenging problem, since it touches many research fields as machine learning, control theory, queueing theory and others.

Moreover, we defined a working plan which would allow us to proceed step by step, incrementally building our results.

References

[1] M. Aldinucci, L. Anardu, M. Danelutto, M. Torquati, and P. Kilpatrick. Parallel patterns + macro data flow for multi-core programming. In *Proceedings of PDP 2012*, pages 27–36, Feb 2012.

[2] M. Aldinucci and M. Danelutto. Skeleton-based parallel programming: Functional and parallel semantics in a single shot. *Computer Languages, Systems and Structures*, 33(3-4):179–192, Oct. 2007.

[3] M. Aldinucci, M. Danelutto, P. Kilpatrick, M. Meneghin, and M. Torquati. Accelerating code on multi-cores with fastflow. In *Proceedings of the 17th International Conference on Parallel Processing - Volume Part II*, Euro-Par'11, pages 170–181. Springer-Verlag, 2011.

[4] M. Aldinucci, M. Danelutto, and M. Vanneschi. Autonomic qos in assist grid-aware components. In *Proceedings of PDP 2006*, pages 10 pp.–.

[5] P. Alonso, M. Dolz, R. Mayo, and E. Quintana-Ort. Modeling power and energy of the task-parallel cholesky factorization on multicore processors. *Computer Science - Research and Development*, 29(2):105–112, 2014.

[6] B. Bacci, M. Danelutto, S. Orlando, S. Pelagatti, and M. Vanneschi. P3L: A structured high level programming language and its structured support. *Concurrency Practice and Experience*, 7(3):225–255, May 1995.

[7] L. Barroso and U. Holzle. The case for energy-proportional computing. *Computer*, 40(12):33–37, Dec 2007.

[8] A. A. Bhattacharya, D. Culler, A. Kansal, S. Govindan, and S. Sankar. The need for speed and stability in data center power capping. In *Proceedings of IGCC 2012*, pages 1–10. IEEE Computer Society.

[9] T. Burd, T. Pering, A. Stratakos, and R. Brodersen. A dynamic voltage scaled microprocessor system. In *ISSCC 2000*, pages 294–295, Feb 2000.

[10] G. Chadha, S. Mahlke, and S. Narayanasamy. When less is more (LIMO):controlled parallelism forimproved efficiency. In *CASES 2012*, page 141. ACM Press, Oct. 2012.

[11] A. Chandrakasan and R. Brodersen. Minimizing power consumption in digital cmos circuits. *Proceedings of the IEEE*, 83(4):498–523, Apr.

[12] C. C. Chi, M. Alvarez-Mesa, and B. Juurlink. Low-power high-efficiency video decoding using general-purpose processors. *ACM Trans. Archit. Code Optim.*, 11(4):56:1–56:25, Jan. 2015.

[13] K. Choi, R. Soma, and M. Pedram. Fine-grained dynamic voltage and frequency scaling for precise energy and performance trade-off based on the ratio of off-chip access to on-chip computation times. In *Proceedings of DATE 2004*, volume 1, pages 4–9 Vol.1, Feb 2004.

[14] R. Cochran, C. Hankendi, A. Coskun, and S. Reda. Identifying the optimal energy-efficient operating points of parallel workloads. In *Proceedings of ICCAD 2011*, pages 608–615. IEEE Press, 2011.

[15] R. Cochran, C. Hankendi, A. K. Coskun, and S. Reda. Pack & cap: Adaptive dvfs and thread packing under power caps. In *Proceedings of MICRO 2011*, pages 175–185. ACM, 2011.

[16] M. Cole. *Algorithmic skeletons: structured management of parallel computation*. MIT Press, 1991.

[17] M. Curtis-Maury, A. Shah, F. Blagojevic, D. S. Nikolopoulos, B. R. de Supinski, and M. Schulz. Prediction models for multi-dimensional power-performance optimization on many cores. In *Proceedings of PACT 2008*, pages 250–259. ACM, 2008.

[18] M. Danelutto, D. De Sensi, and M. Torquati. Energy driven adaptivity in stream parallel computations. In *Proceedings of PDP 2015*, pages 103–110, 2015.

[19] M. Danelutto, L. Deri, D. D. Sensi, and M. Torquati. Deep packet inspection on commodity hardware using fastflow. In *Proceedings of ParCo 2013*, pages 92–99, 2013.

[20] R. Das, M. Harchol-balter, A. Gandhi, J. O. Kephart, and C. Lefurgy. Power capping via forced idleness, 2009.

[21] D. De Sensi. Predicting performance and power consumption of parallel applications. In *Proceedings of PDP 2016*. Under review.

[22] X. Feng, R. Ge, and K. Cameron. Power and energy profiling of scientific applications on distributed systems. In *Proceedings of IPDPS 2005*, pages 34–34, April 2005.

[23] Intel. Desktop 4th Generation Intel Core Datasheet Volume 1 of 2. Section 4 - Processor, Power management. `http://www.intel.it/content/dam/www/public/us/en/documents/datasheets/4th-gen-core-family-desktop-vol-1-datasheet.pdf`.

[24] M. Kim, K. Kim, J. Geraci, and S. Hong. Utilization-aware load balancing for the energy efficient operation of the big.little processor. In *Proceedings of DATE 2014*, pages 1–4, March 2014.

[25] N. Kim, T. Austin, D. Baauw, T. Mudge, K. Flautner, J. Hu, M. Irwin, M. Kandemir, and V. Narayanan. Leakage current: Moore's law meets static power. *Computer*, 36(12):68–75, Dec 2003.

[26] P. Kurp. Green computing. *Commun. ACM*, 51(10):11–13, Oct. 2008.

[27] E. Le Sueur and G. Heiser. Dynamic voltage and frequency scaling: The laws of diminishing returns. In *Proceedings of the 2010 International Conference on Power Aware Computing and Systems*, HotPower'10, pages 1–8. USENIX Association, 2010.

[28] D. Li, B. R. de Supinski, M. Schulz, D. S. Nikolopoulos, and K. W. Cameron. Strategies for energy-efficient resource management of hybrid programming models. *IEEE Transactions on Parallel and Distributed Systems*, 24(1):144–157, 2013.

[29] J. Li and J. Martinez. Dynamic power-performance adaptation of parallel computation on chip multiprocessors. In *Proceedings of HPCA 2006*, pages 77–87, Feb 2006.

[30] E. J. Lucente. The coming "c" change in data centers. `http://www.hpcwire.com/2010/06/15/the_coming_c_change_in_datacenters/`.

[31] D. S. N. Matthew Curtis-maury, Filip Blagojevic, Christos D. Antonopoulos. Prediction-based Power-Performance Adaptation of Multithreaded Scientific Codes. *IEEE TPDS*, 2008.

[32] G. Mencagli, M. Vanneschi, and E. Vespa. A cooperative predictive control approach to improve the reconfiguration stability of adaptive distributed parallel applications. *ACM Trans. Auton. Adapt. Syst.*, 9(1):2:1–2:27, Mar. 2014.

[33] A. Miyoshi, C. Lefurgy, E. Van Hensbergen, R. Rajamony, and R. Rajkumar. Critical power slope: Understanding the runtime effects of frequency scaling. In *Proceedings of ICS 2002*, pages 35–44. ACM.

[34] D. C. Montgomery and E. Peck. *Introduction to linear regression analysis*. Wiley-Interscience Publication. John Wiley & sons, 1992. A Wiley-Interscience publication.

[35] W. J. Morokoff and R. E. Caflisch. Quasi-random sequences and their discrepancies. *SIAM J. Sci. Comput*, 15:1251–1279, 1994.

[36] A. K. Porterfield, S. L. Olivier, S. Bhalachandra, and J. F. Prins. Power Measurement and Concurrency Throttling for Energy Reduction in OpenMP Programs. In *Proceedings of IPDPSW 2013*, pages 884–891. IEEE, May 2013.

[37] K. K. Pusukuri, R. Gupta, and L. N. Bhuyan. Thread reinforcer: Dynamically determining number of threads via os level monitoring. In *Proceedings of IISWC 2011*, pages 116–125. IEEE Computer Society.

[38] P. Ranganathan. Recipe for efficiency: Principles of power-aware computing. *Commun. ACM*, 53(4):60–67, Apr. 2010.

[39] M. A. Suleman, M. K. Qureshi, and Y. N. Patt. Feedback-driven threading: Power-efficient and high-performance execution of multi-threaded workloads on cmps. *SIGARCH Comput. Archit. News*, 36(1):277–286.

[40] E. Totoni, N. Jain, and L. V. Kalé. Power management of extreme-scale networks with on/off links in runtime systems. *TOPC*, 1(2):16, 2015.

[41] E. Totoni, J. Torrellas, and L. V. Kale. Using an adaptive hpc runtime system to reconfigure the cache hierarchy. In *Proceedings of SC 2014*, pages 1047–1058. IEEE Press, 2014.

[42] V. Vassiliadis, K. Parasyris, C. Chalios, C. D. Antonopoulos, S. Lalis, N. Bellas, H. Vandierendonck, and D. S. Nikolopoulos. A programming model and runtime system for significance-aware energy-efficient computing. In *Proceedings of PPoPP 2015*, pages 275–276, 2015.

[43] M. J. I. Yang Ding, Mahmut K, Padma Raghavan. A Helper Thread Based EDP Reduction Scheme for Adapting Application Execution in CMPs.

[44] V. Zyuban, J. Friedrich, C. Gonzalez, R. Rao, M. Brown, M. Ziegler, H. Jacobson, S. Islam, S. Chu, P. Kartschoke, G. Fiorenza, M. Boersma, and J. Culp. Power optimization methodology for the ibm power7 microprocessor. *IBM Journal of Research and Development*, 55(3):7:1–7:9, May 2011.